

새 리소스 시스템 구축 명세 — ResourceRegistry + ResourceScope

계획의 Phase 2(구조 코어)에 해당. 상위 계획: [Refactor_Plan.md](#) §1 목표 구조 / §Phase 2 기준선: [Baseline.md](#) §8 (Baseline v2) 핵심 전환: 2버킷(Global/Scene) + 호출부 isGlobal bool → (key,type) refCount 레지스트리 + 수명 객체(스코프) 성격: 이 단계는 수치 무관 · 동작 보존. 성공 기준 = "실행 왕복 후 잔존 핸들이 Baseline v2와 동일"(안 변해야 성공). 수치를 움직이는 재배포는 이 위에서 도는 Phase 3.

이 문서로 하는 일 / Phase 1 버그를 왜 지금 안 고치나

리소스 관련 내부를 이 새 코어로 전부 이관한다. 기존 `LoadAsyncNoCache` / `Clear` / 2버킷 디렉터리는 사라지고 레지스트리+스코프로 교체된다. 그 과정에서 [Phase1_Spec.md](#)의 버그 4건은 개별 수정 없이 구조적으로 소멸한다(§7 매핑). → 지금 Phase 1을 따로 커밋하지 않고, 이 구축으로 바로 흡수한다.

빅뱅 금지: `Managers.Resource` 파사드와 기존 호출 시그니처(`LoadAsync<T>(key, isGlobal, token)` , 130곳)를 어댑터로 유지한 채 안쪽만 교체. 호출부 일괄 수정 없음.

1. 설계 3원칙 (여기서 벗어나면 리팩터의 의미가 없음)

1. 핸들 소유권은 레지스트리 한 곳. 스코프는 '참조 티켓'만 든다 → "누가 Release하나" 문제 자체가 소멸(HP바 누수의 근본 원인).
2. 스코프 내 중복 로드 = refCount 1. 같은 스코프가 같은 키를 N번 로드해도 `HashSet.Add` 반환값으로 1회만 카운트 → 호출부가 로드 횟수를 셀 필요 없음.
3. 취소 ≠ 해제. 취소 토큰은 '기다림 중단'만. 핸들 수명은 스코프가 소유 → `await` 후 파괴된 에셋 접근(use-after-release)이 구조적으로 불가능.

2. 목표 구조

ResourceManager (파사드 — 기존 API 유지)

- └ 새 API : `scope.LoadAsync<T>(key, token)` ← 수명은 스코프가 소유
- └ 어댑터 : `LoadAsync<T>(key, isGlobal, token)` ← 기존 130곳 그대로 (isGlobal ? Global : Scene)
- └ `GlobalScope` (앱 수명, 절대 Dispose 안 함)
- └ `SceneScope` (현재 씬, 전환 시 Rotate)
- └ `CreateScope(name) / RotateSceneScope()`

ResourceRegistry ← 핸들을 소유하는 유일한 곳

- └ `Dictionary<ResourceKey, Entry{ handle, refCount }>`
- 키+타입당 핸들 1개 (타입충돌 핵 P5 소멸)
- 로딩 중이면 같은 핸들을 공유 대기 (중복 전개 소멸)
- refCount 0이 되는 순간에만 `Addressables.Release`

ResourceScope : `IDisposable` ← 참조 티켓(`HashSet<ResourceKey>`)만 보유

- Global
 - └ Scene (BaseScene/전환에 배선)
 - └ `Dungeon/Popup` 등 (using 블록 or 명시적 Dispose) ← Phase 3에서 활용
- Dispose → 보유 키 전부 `registry.Release`

3. 구성요소 명세

파일 위치 제안: `Assets/Scripts/Managers/Core/Resource/` 하위에 `ResourceKey.cs` · `ResourceRegistry.cs` · `ResourceScope.cs` 신설, `ResourceManager.cs` 는 기존 위치에서 내부만 교체.

3.1 ResourceKey — (key, type) 복합 키

```
public readonly struct ResourceKey : IEquatable<ResourceKey>
{
    public readonly string Key;
    public readonly Type Type;
    public ResourceKey(string key, Type type) { Key = key; Type = type; }

    public bool Equals(ResourceKey o) => Key == o.Key && Type == o.Type;
    public override bool Equals(object o) => o is ResourceKey k && Equals(k);
    public override int GetHashCode() => (Key, Type).GetHashCode();
    public override string ToString() => $"{Key}<{Type.Name}>";
}
```

- 왜 타입까지: 같은 문자열 키를 `GameObject` / `Sprite` 등 다른 T로 로드해도 서로 다른 엔트리 → 기존 "타입 불일치 시 핸들 빼서 재로드"하던 증상치료 핵(P5)이 근본 소멸.

3.2 `ResourceRegistry` — 핸들 소유 · refCount · 로딩중 공유

```

public sealed class ResourceRegistry
{
    private sealed class Entry
    {
        public AsyncOperationHandle Handle;
        public int RefCount;
    }

    private readonly Dictionary<ResourceKey, Entry> _entries = new();

    // incrementRef=true 일 때만 refCount++ (스코프가 '이 스코프의 첫 참조'일 때만 true로 호출)
    public async UniTask<T> LoadAsync<T>(ResourceKey key, bool incrementRef, CancellationToken token)
        where T : UnityEngine.Object
    {
        if (_entries.TryGetValue(key, out var e))
        {
            if (incrementRef) e.RefCount++;
            if (!e.Handle.IsDone) // 이미 로딩 중 → 같은 핸들 공유 대기
                await e.Handle.ToUniTask(cancellationToken: token);
            return e.Handle.Result as T;
        }

        var handle = Addressables.LoadAssetAsync<T>(key.Key);
        e = new Entry { Handle = handle, RefCount = incrementRef ? 1 : 0 };
        _entries[key] = e; // await '전에' 등록 → 동시 요청이 같은 핸들 공유
        await handle.ToUniTask(cancellationToken: token); // 취소돼도 핸들은 여기 소유로 유지 (원칙 3)

        if (handle.Status != AsyncOperationStatus.Succeeded)
        {
            GameLog.LogError($"[ResourceRegistry] Load Failed: {key}");
            _entries.Remove(key); // 실패 엔트리 제거
            return null;
        }
        return handle.Result as T;
    }

    public void Release(ResourceKey key)
    {
        if (!_entries.TryGetValue(key, out var e)) return;
        if (--e.RefCount <= 0)
        {
            if (e.Handle.IsValid()) Addressables.Release(e.Handle);
            _entries.Remove(key);
        }
    }

    // 프리로드된 에셋 동기 조회 (Instantiate(key) 경로용)
    public bool TryGetAsset<T>(string key, out T asset) where T : UnityEngine.Object
    {
        asset = null;
        if (_entries.TryGetValue(new ResourceKey(key, typeof(T)), out var e)
            && e.Handle.IsDone && e.Handle.Result is T t) { asset = t; return true; }
        return false;
    }
}

```

주의 · 엣지 케이스(구현 시 확인):

- 취소: `await` 가 `OperationCanceledException` 을 던져도 엔트리는 남는다. `refCount`는 이미 올랐고 스코프 티켓에도 들어가 있으므로 → 스코프 Dispose 때 정상 회수. 호출부는 취소를 함수(기존 `InstantiateAsync` 의 `catch(OperationCanceledException)` 패턴 유지).
- 로딩 중 실패: 실패 시 엔트리를 지우지만, 그 사이 `refCount`를 올린 스코프의 티켓엔 키가 남는다 → Dispose 때 `Release` 가 없는 키를 만나면 `TryGetValue` 가드로 no-op(누수 아님). 티켓 정리는 선택.
- 동시 로드: 같은 키를 두 곳이 거의 동시에 요청 → 첫 호출이 엔트리를 `await` 전에 등록하므로 둘째는 `IsDone==false` 분기로 같은 핸들을 공유 대기. 중복 전개(AtlasMetric 4~6회) 소멸.

3.3 ResourceScope — 참조 티켓, IDisposable

```
public sealed class ResourceScope : IDisposable
{
    private readonly ResourceRegistry _registry;
    private readonly HashSet<ResourceKey> _acquired = new();
    private bool _disposed;
    public string Name { get; }

    public ResourceScope(ResourceRegistry registry, string name)
    { _registry = registry; Name = name; }

    public UniTask<T> LoadAsync<T>(string key, CancellationToken token = default)
        where T : UnityEngine.Object
    {
        if (_disposed || string.IsNullOrEmpty(key)) return UniTask.FromResult<T>(null);

        var rk = new ResourceKey(key, typeof(T));
        bool firstRefFromThisScope = _acquired.Add(rk); // 이 스코프가 처음일 때만 true (원칙 2)
        return _registry.LoadAsync<T>(rk, incrementRef: firstRefFromThisScope, token);
    }

    public void Dispose()
    {
        if (_disposed) return;
        _disposed = true;
        foreach (var rk in _acquired) _registry.Release(rk);
        _acquired.Clear();
    }
}
```

- 중복 로드 무비용: `HashSet.Add` 가 false면 registry엔 `incrementRef:false` 로 가서 refCount를 안 올림 → 스코프당 키 1참조 보장.

3.4 ResourceManager 파사드 — 어댑터 + 스코프 소유

```

public ResourceScope GlobalScope { get; private set; }
public ResourceScope SceneScope { get; private set; }
private ResourceRegistry _registry;

public void Init()
{
    _registry = new ResourceRegistry();
    GlobalScope = new ResourceScope(_registry, "Global");
    SceneScope = new ResourceScope(_registry, "Scene");
}

// —— 어댑터: 기존 130개 호출부 시그니처 무수정 ——
public UniTask<T> LoadAsync<T>(string key, bool isGlobal = false, CancellationToken token = default)
    where T : UnityEngine.Object
    => (isGlobal ? GlobalScope : SceneScope).LoadAsync<T>(key, token);

// AssetReference 오버로드도 동일하게 위임 (기존 시그니처 유지)
public UniTask<T> LoadAsync<T>(AssetReference r, bool isGlobal = false, CancellationToken token = default)
    where T : UnityEngine.Object
    => (r == null || !r.RuntimeKeysValid())
        ? UniTask.FromResult<T>(null)
        : LoadAsync<T>(r.RuntimeKey.ToString(), isGlobal, token);

// —— 신설 ——
public ResourceScope CreateScope(string name) => new ResourceScope(_registry, name);

public void RotateSceneScope() // 씬 전환용: 새 스코프 준비 후 이전 것 Dispose
{
    var old = SceneScope;
    SceneScope = new ResourceScope(_registry, "Scene");
    old.Dispose();
}

```

- 기존 `Clear()` 는 `RotateSceneScope()` 로 대체(의미: 씬 스코프 회수). 다만 호출 시점이 P3의 핵심 → §5.

4. 130개 호출부 어댑터 전략

- 대부분은 `Managers.Resource.LoadAsync<T>(key, isGlobal)` 그대로 → 무수정(어댑터가 스코프로 라우팅).
- 새 코드/재배치 대상만 명시적 스코프 사용(`CreateScope` 또는 던전/팝업 스코프). 이걸 Phase 3에서 점진 전환.
- 목표: 이 단계 끝에 호출부 diff는 거의 0, 바뀐 건 `ResourceManager.cs` 내부 + 씬 전환 배선(§5) + JSON/Preload/Instantiate 내부 경로(§6)뿐.

5. 씬 전환 배선 (P3 계약 = 해제 창 제거)

계약: 이전 씬 스코프의 Dispose는 반드시 새 씬(=Loading 씬) 진입 완료 후. 렌더 중 해제 창·이중 Clear 제거.

- `BaseScene.Clear()` — 기존 `Managers.Resource.Clear()` 한 줄 제거. (서브클래스의 이벤트 해제·사운드 정지 등 고유 정리는 `base.Clear()` 가 비어도 그대로 실행 → 무해)
- `LoadingScene.LoadProcessAsync` — 기존 `Managers.Resource.Clear()` (L39)를 `Managers.Resource.RotateSceneScope()` 로 교체. 이 시점은 Loading 씬이 이미 활성화이라 이전 씬 렌더 창이 없음.
- 검증 혹은 유지: 회전 직후 `AssertSceneHandlesCleared` 대신 레지스트리 기준으로 "씬 스코프 티켓 비었나 / 씬 전용 키가 refCount 0으로 회수됐나"를 보게 디버그 창을 갱신(§6-e).

모든 씬 전환은 `SceneManagerEx.LoadSceneAsync` → Loading 씬 단일 경로 → `RotateSceneScope` 가 반드시 1회 실행됨(누수 없음).

6. 특수 경로 통합

a. NoCache / JSON (P2 소멸)

`LoadAsyncNoCache` 삭제. 임시 스코프 + 값 복사로 대체 — 소비가 끝난 뒤 Dispose라 use-after-release 불가.

```
// ResourceManager
public async UniTask<string> LoadTextAsync(string key)
{
    using var scope = CreateScope("TempText");
    var asset = await scope.LoadAsync<TextAsset>(key);
    return asset != null ? asset.text : null; // string 값 복사 → Dispose로 핸들 해제해도 안전
} // using 종료 = scope.Dispose() = registry.Release
```

`DataManager.LoadAndCacheJsonAsync` 는 `var json = await Managers.Resource.LoadTextAsync(key);` 후 `JsonUtility.FromJson` .

b. 프리로드

`LoadDependenciesAsync(labels, ResourceScope scope, onProgress)` — location마다 `scope.LoadAsync<Object>(key)` 로 획득.
어댑터 `LoadDependenciesAsync(labels, isGlobal, ...)` = `isGlobal ? GlobalScope : SceneScope` . (순차 로드 P6는 개선 축 아님 — 구조만 스코프로 옮기고 배치하는 Phase 5 선택.)

c. Instantiate(string key)

레지스트리 동기 조회로 교체: `if (_registry.TryGetAsset<GameObject>(key, out var prefab)) return Instantiate(prefab, ...);` — "미리 LoadAsync 해두었는지" 계약은 그대로.

d. 아틀라스 스프라이트 (`GetSpriteFromAtlasAsync`)

아틀라스 핸들을 스코프로 획득하도록만 전환(현재 `LoadAsync<SpriteAtlas>(atlasKey, isGlobal:true)` → 스코프 경유).
`_atlasSpriteCache` (현재 해제 경로 없는 영구 캐시)를 아틀라스 핸들 수명에 동행시키는 건 Phase 3d 몫 — 이 단계에선 동작 보존만(캐시 유지). 0.5c에서 대형 아틀라스는 이미 해제돼 개별 `LoadAsync<Sprite>` 라 잔여 소비는 아이콘류뿐.

e. 디버그 창 (영구 자산 유지)

`ResourceDebugWindow` / `LogAliveReport` / 스냅샷이 2버킷 대신 레지스트리 엔트리(키·타입·refCount·소유 스코프)를 가리키도록 갱신. Phase 0 계측 도구는 버리지 않고 레지스트리를 보게 이어붙인다.

7. Phase 1 버그 4건이 자동 소멸하는 방식

버그 (Phase1_Spec)	새 구조에서 왜 사라지나
① NoCache use-after-release	<code>LoadTextAsync</code> 가 임시 스코프에서 <code>.text</code> 복사 후 Dispose → 소비 후 해제로 계약 자체가 바뀜
② 캐시 히트 승격 누락	"isGlobal"이 조회 분기가 아니라 어느 스코프가 acquire하나로 바뀜. Global·Scene이 같은 키를 참조하면 refCount 2, 둘 다 Dispose돼야 해제 → 승격 개념 자체가 불필요
③ 해제 순서 + 이중 Clear	<code>RotateSceneScope</code> 가 "새 스코프 준비 후 이전 Dispose" 순서를 강제 + 단일 호출 → 렌더 창·이중 호출 소멸(\$5)
④ <code>_isLoadingPopup</code> 미복구	이건 UIManager 로직 버그라 구조로 안 사라짐 → 팝업을 스코프화하는 Phase 3a에서 <code>ShowPopupUIAsync</code> 를 손볼 때 <code>try/finally</code> 로 함께 정리(Phase1_Spec §4 참고). 그 전까지는 무해하게 잔존

①②③은 구축과 함께 사라지고, ④만 Phase 3a에서 흡수. 따라서 지금 Phase 1을 따로 커밋할 실익이 없음.

8. 구현 순서 (각 단계 = 독립 커밋 = 롤백 지점)

- 코어 3종 신설 — `ResourceKey` · `ResourceRegistry` · `ResourceScope` (아직 미배선, 컴파일만). 커밋: `리소스 Phase 2 — 레지스트리/스코프 코어 신설`

2. 파사드 내부 교체 — `ResourceManager` 가 2버킷 딕셔너리 제거, 레지스트리+Global/Scene 스코프 보유, 어댑터 `LoadAsync(key,isGlobal)` 라우팅. `Instantiate(key)` 를 `TryGetAsset` 으로. 130 호출부 무수정 확인. 커밋: — 파사드 어댑터 배선
 3. 씬 전환 배선 — `BaseScene.Clear` 정리 + `LoadingScene` → `RotateSceneScope` (§5). 커밋: — 씬 스코프 회전
 4. JSON/프리로드 경로 — `LoadTextAsync` + `DataManager` 전환, `LoadDependenciesAsync` 스코프화 (§6a,b). 커밋: — NoCache/프리로드 스코프화
 5. 디버그 창 이관 — 스냅샷/리포트를 레지스트리 기준으로 (§6e). 커밋: — 계측 도구 레지스트리 연결
- 각 커밋 후: 에디터(또는 배치) 컴파일 → 디버그 창으로 씬 왕복·팝업 개폐 → 잔존 핸들이 Baseline v2와 동일한지 확인.

9. 검증 (동작 보존 = 이 단계의 성공 정의)

- 불변 지표(성공 조건): 씬 왕복 후 잔존 핸들 수/키 목록 = [Baseline.md §8](#)와 동일. 진입 Global 512.8MB, 팝업 세션 +46.8MB, 왕복 +18.7MB 그대로(아직 재배포 전).
- 개선 지표: [\[AtlasMetric\]](#) 중복 전개 4~6회 → 소멸(로딩중 공유). 이걸 이 단계에서 유일하게 "좋아지는" 항목(예측표 P2열, [Refactor_Plan §2.5](#)).
- 회귀 없음: 팝업 개폐·씬 전환 중 핑크 텍스처/누락 없음(육안). [\[ResourceLeak\]](#) 류 assert 미발생.
- 측정 절차 공통: 플레이 → 디버그 창 체크포인트 버튼 ①~⑤ → [MetricsLogs/*.log](#) .

10. 범위 밖 / 다음

- Phase 3(수명 재배포, 메인): 팝업=씬 스코프, 대형 아틀라스=소비 UI 스코프, Global 프리로드 다이어트, 던전 스코프 — 여기서 비로소 모든 수치가 움직인다. 이 구축은 그걸 누수 없이 가능하게 하는 전제조건.
- Phase 4: 풀↔에셋 수명 통합(PoolManager를 스코프 소속으로, DDOL 루트 제거). PoolManager는 현재 인스턴스만 들고 에셋 핸들 ref를 안 잡음(P4) — 이 단계에선 손대지 않음.
- ④ `_isLoadingPopup` : Phase 3a에서 팝업 스코프화와 함께 `try/finally` 정리.